

System Analysis & Design

Movie Search Application — Part 1

1. Problem Statement & Objectives

Problem Statement

Most people search for movies online but end up jumping between multiple websites to find basic information like the plot, rating, or cast. There is no single simple interface that lets you search and immediately see all the details you need in one place.

This project solves that by building a clean React app that connects to a Movie Database API — the user types a movie name, gets results instantly, and can click any movie to see all its details on one page.

Project Objectives

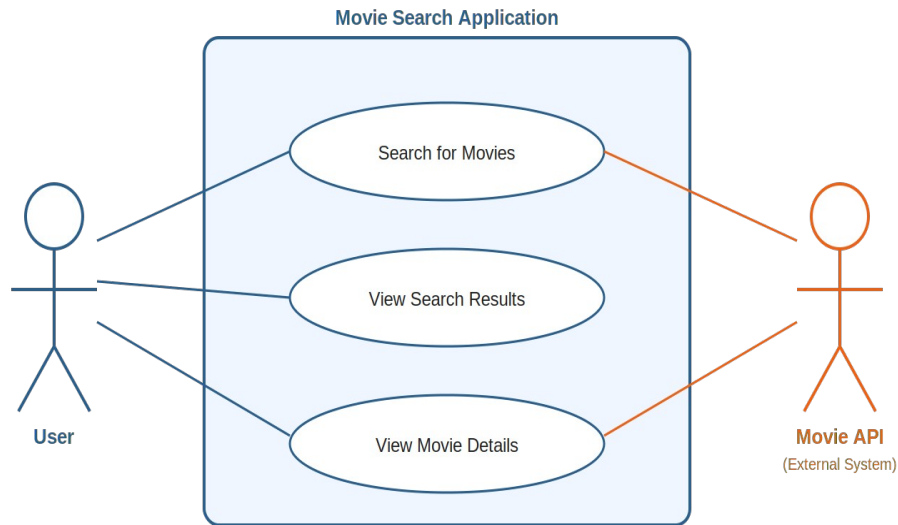
- Build a Search Bar that fetches real movie results from an external API
- Build a Movie Details page that shows title, poster, rating, genres, and plot
- Make the app responsive so it works on both desktop and mobile
- Handle loading and error states so the user always sees a clear message
- Write clean, well-structured React code using components and hooks

Goals Summary

Goal	Description
Functional	Search works, details page works, navigation works
Usability	Simple UI — any user can use it without instructions
Performance	Results appear in under 2 seconds
Code Quality	Clean components, no hardcoded API keys, good error handling

2. Use Case Diagram

The diagram below shows the two main actors (User and Movie API) and the four use cases in the system.



Use Case Descriptions

Use Case	Actor	Description
Search for Movies	User	User types a keyword in the search bar and the app fetches matching movies from the Movie API
View Search Results	User	User sees the list of movies returned by the search, displayed as cards with poster, title, and year
View Movie Details	User	User clicks a movie card and the app fetches and displays full details (plot, rating, genres, cast)

3. Functional & Non-Functional Requirements

Functional Requirements

ID	Requirement	Priority
FR-01	Search bar on the home page for keyword-based movie search	High
FR-02	Search triggers on Enter key or Search button click	High
FR-03	App calls Movie Database API and displays results as movie cards	High
FR-04	Each card shows: poster, title, release year	High
FR-05	Clicking a card opens the Movie Details page	High
FR-06	Details page shows: title, poster, plot, rating, genres, release date	High
FR-07	Back button returns user to search results	High
FR-08	Empty search shows a validation message	Medium
FR-09	No results returns a 'No results found' message	Medium

FR-10	Loading spinner shown while API call is in progress	Medium
FR-11	Pagination controls shown when results span multiple pages	Medium

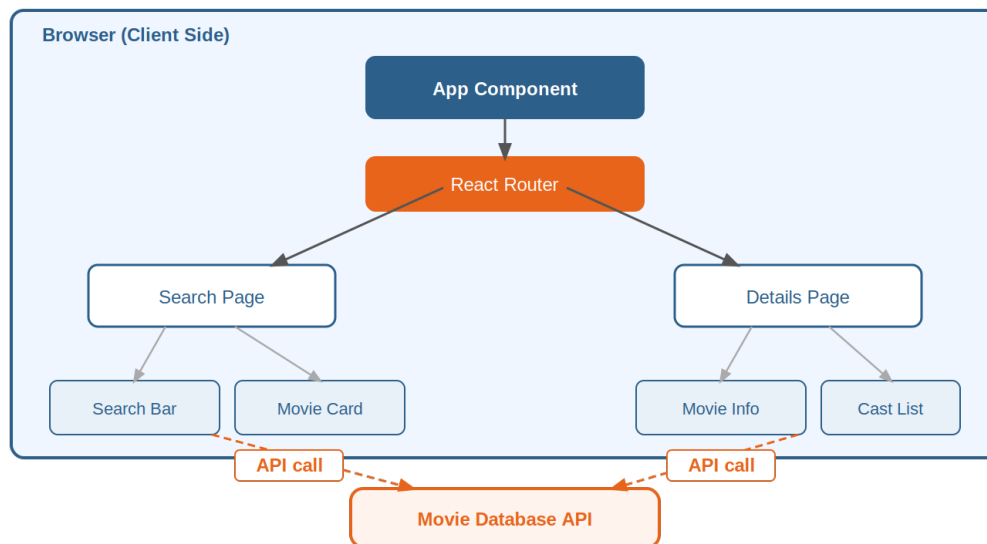
Non-Functional Requirements

Category	Requirement
Performance	Search results must appear within 2 seconds under normal connection
Security	API key stored in .env file — never in the source code
Usability	UI must work correctly on screen sizes from 320px to 1920px
Reliability	App must not crash on API failure — always show an error message
Compatibility	Must work on Chrome, Firefox, and Edge browsers

4. Software Architecture

The app follows a Component-Based Architecture, which is the standard approach for React applications. There is no backend — the app runs entirely in the browser and communicates directly with the Movie Database API.

Component Architecture — Movie Search App (React SPA)



Architecture Layers

Layer	Technology	What it Does
UI Layer	React Components	Renders all pages and handles user interactions
Routing Layer	React Router	Handles navigation between Search page and

		Details page
State Layer	React useState / useEffect	Manages search query, results, loading, and error states
Data Layer	Movie Database API (REST)	External source of all movie data — called via fetch()

Key Architecture Decisions

- No backend needed — React SPA calls the API directly from the browser
- React Router handles page navigation without full page reload
- Each feature is its own component (SearchBar, MovieCard, MovieDetails...)
- API key protected using environment variables (.env file)

Database Design & Data Modeling

Movie Search Application — Part 2

1. Data Source Overview

This application does not use a traditional database. All movie data comes directly from an external Movie Database API. The app sends HTTP requests to the API and receives JSON responses — no data is stored locally.

Because of this, there is no need for an ER Diagram, database tables, or normalization. Instead, this section documents the structure of the data returned by the API.

How Data Flows

- User types a search query in the Search Bar
- The app sends an API request with the query (e.g., `/search?q=batman`)
- The API returns a JSON response with a list of matching movies
- When the user clicks a movie, the app sends another request (e.g., `/movie/123`)
- The API returns a JSON response with full details for that movie

2. API Data Structures

Below are the main data objects the app receives from the Movie Database API.

Search Result (Movie List Item)

When the user searches for a movie, the API returns a list of results. Each item in the list contains:

Field	Type	Description	Example
id	Number	Unique movie identifier	550
title	String	Movie title	Fight Club
poster	String (URL)	URL of the movie poster image	https://image.api.com/poster.jpg
release_date	String	Release date of the movie	1999-10-15
overview	String	Short plot summary	An insomniac office worker...

Movie Details Object

When the user clicks a movie card, the app fetches full details. The response contains:

Field	Type	Description	Example
id	Number	Unique movie identifier	550

title	String	Movie title	Fight Club
poster	String (URL)	Full poster image URL	https://image.api.com/poster.jpg
backdrop	String (URL)	Background banner image URL	https://image.api.com/bg.jpg
overview	String	Full plot description	An insomniac office worker...
release_date	String	Release date	1999-10-15
rating	Number	Average user rating (0-10)	8.4
runtime	Number	Movie length in minutes	139
genres	Array	List of genre names	Drama, Thriller
cast	Array	Top billed actors and their roles	Brad Pitt as Tyler Durden

3. Summary

Question	Answer
Does the app use a database?	No — all data comes from an external API
Where is data stored?	On the API server — the app only reads it
What format is the data?	JSON — received via HTTP GET requests
Does the app save anything locally?	No — no localStorage, no cookies, no database

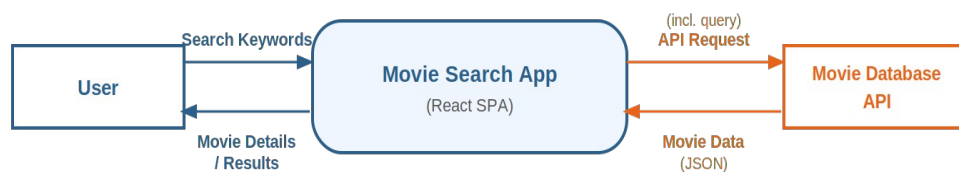
Data Flow & System Behavior

Movie Search Application — Part 3

1. Data Flow Diagram (DFD)

The DFD shows how data moves between the User, the App, and the Movie Database API. This is a context-level diagram — it shows the system as one process.

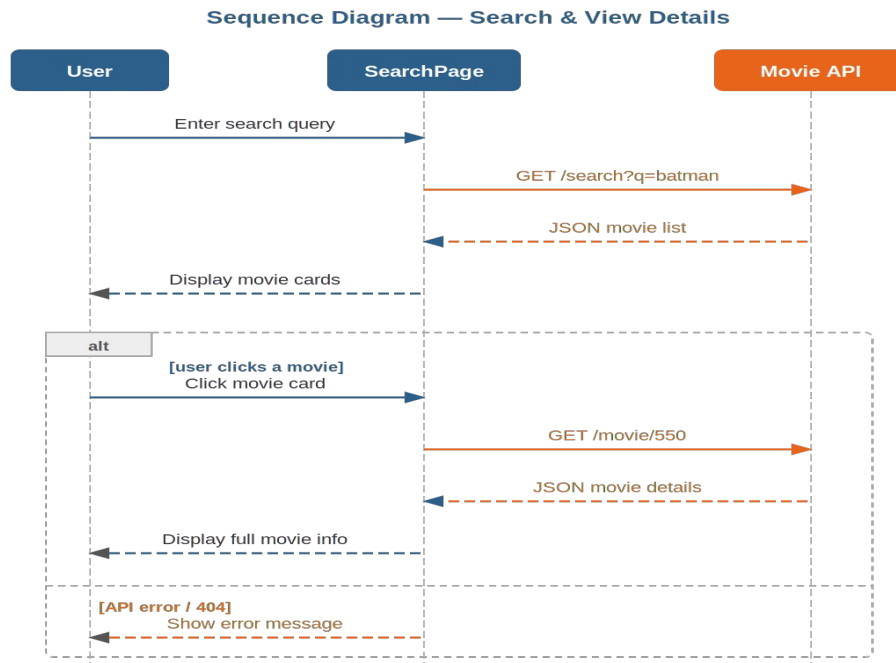
Context-Level DFD — Movie Search App



The User sends a search query to the app. The app sends an API request to the Movie Database API. The API returns JSON data, and the app displays the results to the User.

2. Sequence Diagram

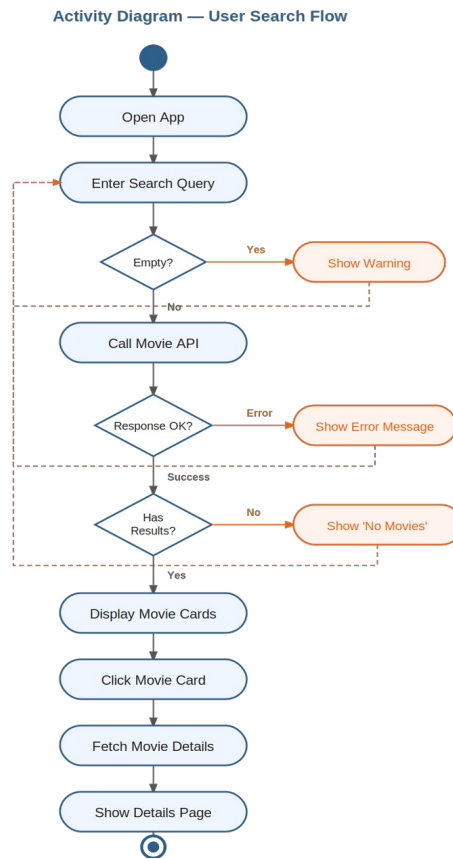
The sequence diagram shows the step-by-step interaction between the User, the Search Page, the Details Page, and the Movie API — in the order they happen.



The flow has two main parts: searching for movies (steps 1–4) and viewing movie details (steps 5–8). Both parts involve an API call and a response displayed to the user.

3. Activity Diagram

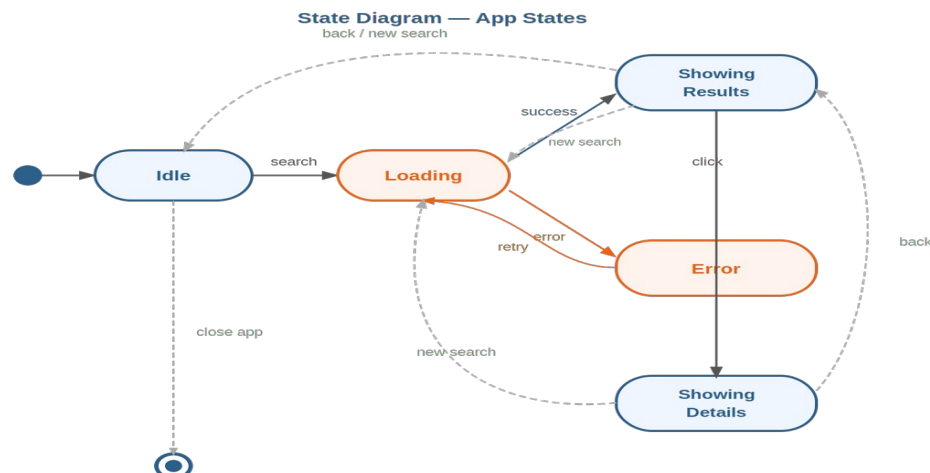
The activity diagram shows the workflow of actions the user goes through when using the app — from opening it to viewing movie details.



If the search box is empty, the app shows a warning and loops back. Otherwise, it calls the API, displays results, and lets the user click through to details.

4. State Diagram

The state diagram shows the different states the app can be in and how it moves between them.

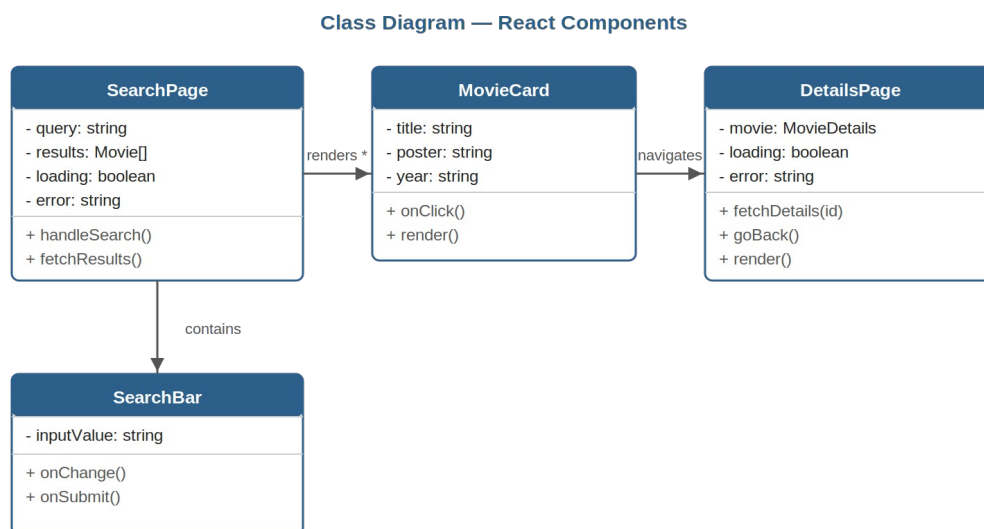


States Explained

- **Idle** — The app is waiting for the user to type a search query
- **Loading** — The app is waiting for a response from the Movie API
- **Showing Results** — Search results are displayed on screen
- **Showing Details** — Full movie details are displayed on screen
- **Error** — The API call failed and an error message is shown

5. Class Diagram

The class diagram shows the main React components in the app, their properties (state), their methods, and how they relate to each other.



Components Explained

- SearchPage — The main page with the search bar and results grid
- SearchBar — The text input where the user types their movie query
- MovieCard — A single movie card showing poster, title, and year
- DetailsPage — The page that shows full details for a selected movie

UI/UX Design & Prototyping

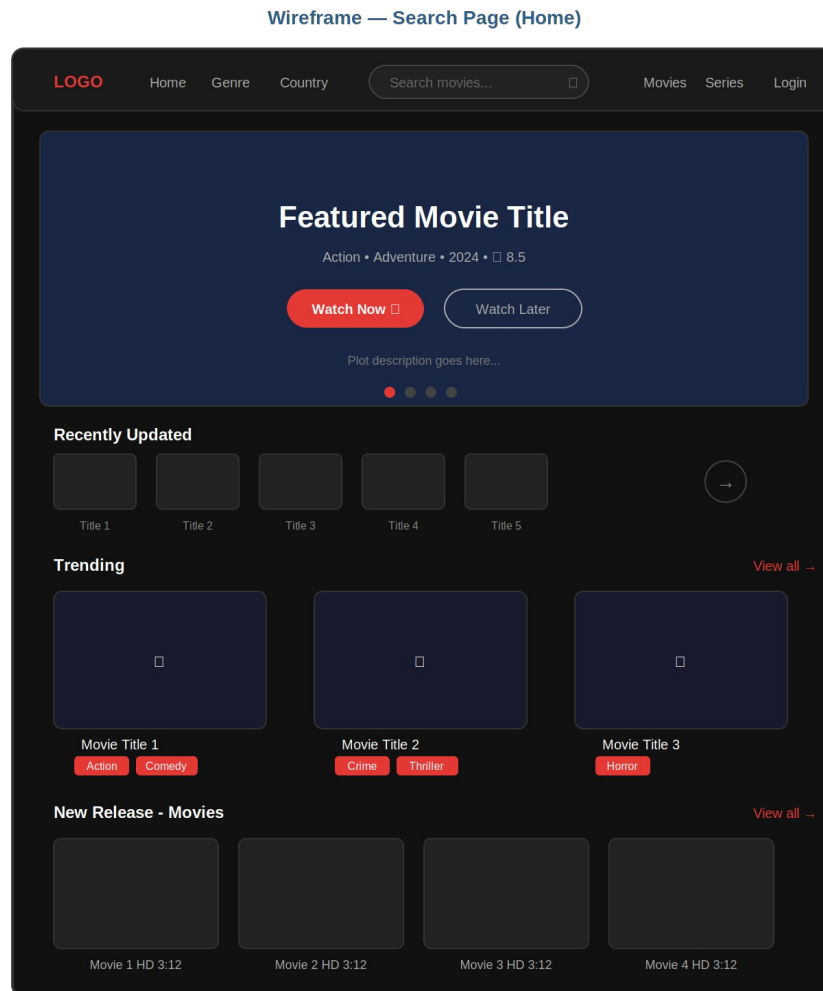
Movie Search Application — Part 4

1. Wireframes & Mockups

The wireframes below show the planned layout for the two main pages of the app. The design uses a dark theme inspired by popular streaming platforms.

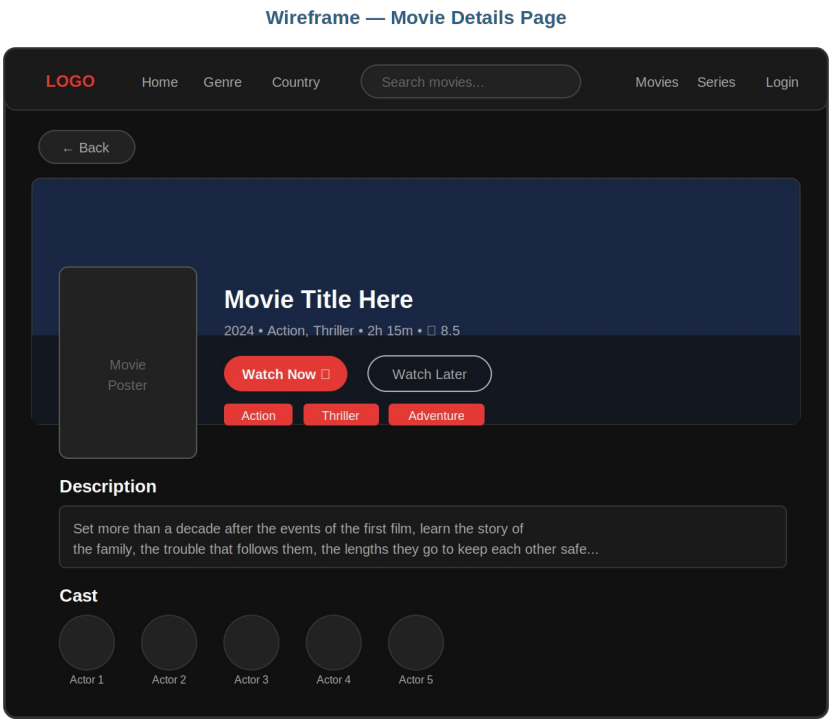
Search Page (Home)

The home page includes a top navigation bar with search, a hero banner for a featured movie, and sections for Recently Updated, Trending, and New Releases — each with horizontal movie cards.



Movie Details Page

When the user clicks a movie card, they are taken to the details page. It shows a backdrop image, movie poster, title, metadata (year, genre, rating, runtime), a description, and the cast section.



2. UI/UX Guidelines

Color Scheme

Element	Color	Hex Code
Background (main)	Dark Black	#111111
Navbar / Cards	Dark Gray	#1A1A1A
Primary Accent	Red	#E53935
Text (primary)	White	#FFFFFF
Text (secondary)	Light Gray	#AAAAAA
Card Background	Dark Navy	#1A1A2E
Borders	Subtle Gray	#333333

Typography

Element	Font	Size	Weight
Page Title / Hero	Arial / Sans-serif	22–28px	Bold

Section Headings	Arial / Sans-serif	16–18px	Bold
Movie Title	Arial / Sans-serif	14–16px	Bold
Body Text / Metadata	Arial / Sans-serif	12–14px	Normal
Genre Tags	Arial / Sans-serif	10–12px	Normal

Layout & Spacing

- Navbar: Fixed at top, height 50px, dark background with logo on left and search in center
- Hero Banner: Full width, ~250px height, with movie info overlay and action buttons
- Movie Cards: Grid layout — 3 to 4 cards per row on desktop, 2 on tablet, 1 on mobile
- Card Size: ~200px wide x 130px tall for landscape, ~120px x 180px for portrait posters
- Spacing between cards: 16px gap
- Section padding: 24px on each side

Responsive Design

Screen Size	Breakpoint	Layout Changes
Desktop	1024px and above	4 cards per row, full navbar, hero banner visible
Tablet	768px – 1023px	2–3 cards per row, search bar still visible
Mobile	Below 768px	1–2 cards per row, hamburger menu, stacked layout

Accessibility

- All images have alt text describing the movie poster
- Buttons and links are keyboard-navigable (Tab + Enter)
- Minimum color contrast ratio of 4.5:1 for text on dark backgrounds
- Focus indicators visible on interactive elements
- Genre tags use sufficient contrast (white text on red background)

System Deployment & Integration

Movie Search Application — Part 5 & 6

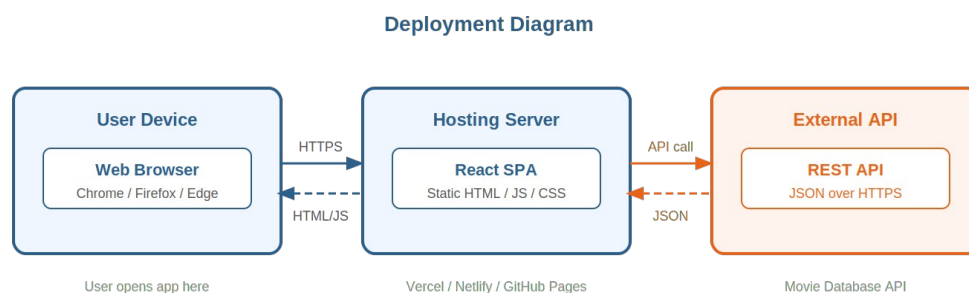
1. Technology Stack

The app is a frontend-only React SPA. There is no backend or database — all data comes from an external API.

Layer	Technology	Purpose
Framework	React.js	Build UI components and manage state
Build Tool	Vite	Fast project setup and dev server
Routing	React Router	Client-side page navigation
Styling	CSS Modules	Scoped per-component styles
Data Source	Movie Database API	External REST API for movie data
Version Control	GitHub	Code hosting and team collaboration
Deployment	Vercel / Netlify	Free static site hosting

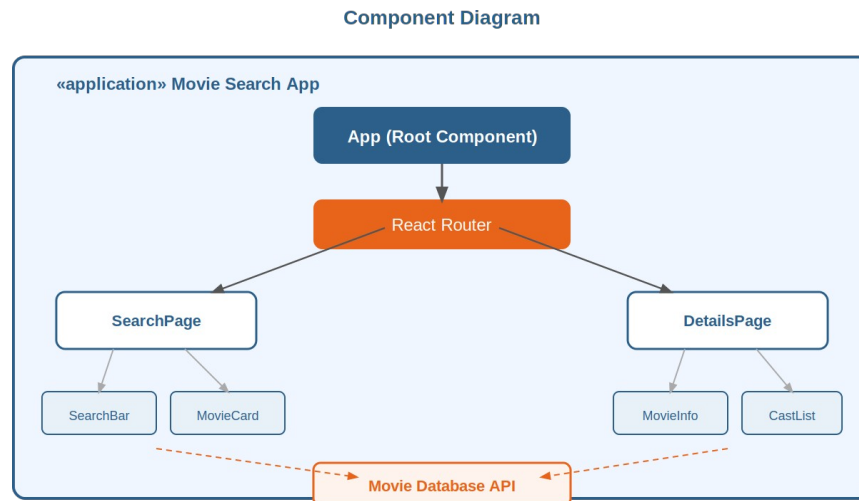
2. Deployment Diagram

The diagram below shows how the app is distributed across three nodes: the user's browser, the hosting server, and the external API.



3. Component Diagram

This diagram shows the high-level React components and their dependencies. Both SearchPage and DetailsPage communicate with the Movie Database API.



4. API Documentation

The app uses two main API endpoints to fetch movie data:

Endpoints Used

Method	Endpoint	Description	Parameters
GET	/search/movie	Search for movies by keyword	query (string) — the search term
GET	/movie/{id}	Get full details for a specific movie	id (number) — the movie ID

Example Request & Response

Search: GET /search/movie?query=batman

- Returns: list of movies with id, title, poster, release_date, overview

Details: GET /movie/550

- Returns: full movie object with title, poster, backdrop, overview, genres, rating, runtime, cast

Authentication

- API key is required for all requests
- Key is stored in .env file — never exposed in client-side code
- Key is passed as a query parameter or in the request header

5. Testing Plan

The following test scenarios will be used to verify the app works correctly:

ID	Test Case	Expected Result	Type
TC-01	Search with a valid keyword (e.g., 'batman')	Movie cards are displayed with correct data	Functional
TC-02	Search with empty input	Validation message is shown	Functional
TC-03	Search with a keyword that has no results	'No movies found' message is shown	Functional
TC-04	Click a movie card	Details page opens with correct movie info	Functional
TC-05	Click the back button on details page	Returns to search results	Functional
TC-06	API fails or times out	Friendly error message is shown	Error Handling
TC-07	Open app on a mobile screen (375px)	Layout is responsive and usable	Usability
TC-08	Open app on Chrome, Firefox, Edge	App works correctly on all browsers	Compatibility

6. Deployment Strategy

Hosting

- The app will be deployed as a static site on Vercel, Netlify, or GitHub Pages
- All three platforms offer free hosting for static React apps
- Deployment is automatic — push to GitHub triggers a new build

Build Process

- Run 'npm run build' to create an optimized production build
- Vite compiles the React code into static HTML, JS, and CSS files
- The output folder (dist/) is uploaded to the hosting platform

Environment Variables

- API key stored in a .env file at the project root
- The .env file is added to .gitignore so it is never pushed to GitHub
- On the hosting platform, the API key is set as an environment variable in the dashboard